Procesadores de Lenguajes.
Universidad Autónoma de Occidente.
Ingeniería de Software Grupo “04” J303.
Matrículas 24040618 y 24040780.

Semana 0. Visión del producto y backlog inicial

Profesora:
Nidia Deniss Lugo.

Alumnos:
Ebrahim Ali.
Rey David Ayala Noriega.

Culiacán, doce de noviembre de 2026.

1. Introducción

Para este proyecto de Procesadores de Lenguajes tenemos que crear nuestro propio metalenguaje, que es un lenguaje que sirve para describir otras cosas, y hacerlo usando Scrum durante cuatro sprints de una semana. Este documento es la entrega de la Semana 0, o sea la preparación que se hace antes de empezar el primer sprint. Aquí explicamos quiénes formamos el equipo y qué rol tiene cada uno, qué lenguaje vamos a construir y para quién, el backlog inicial con las historias de usuario y la Definition of Done que vamos a usar para decidir cuándo algo está terminado. Al final tambien declaramos en qué usamos herramientas de inteligencia artificial, como lo pide el enunciado.

2. Equipo y roles

Por ahora el equipo es de dos integrantes, pero dejamos un lugar abierto porque es posible que se una otra persona. Si eso pasa, la agregamos a la tabla y repartimos el trabajo desde el siguiente sprint. Aunque cada quien tiene un rol, los dos vamos a programar y hacer pruebas.

Integrante	Rol Scrum	Qué hace
Ebrahim Ali	Product Owner y Development Team	Mantiene el Product Backlog ordenado por prioridad, escribe los criterios de aceptación y representa a la docente, que en este proyecto es la "cliente".
Rey David Ayala Noriega	Scrum Master y Development Team	Organiza el Planning, los Daily, la Review y la Retrospective, y se encarga de quitar lo que esté frenando al equipo.
(Lugar disponible)	Development Team	Diseñar, programar y probar el metalenguaje.

Tabla 1. Integrantes del equipo y roles Scrum.

3. Visión del producto

3.1 El problema

Cuando un docente hace un examen normalmente lo escribe en Word o lo arma directo en una plataforma. Si después quiere usar las mismas preguntas en otro lugar, como una página o un formulario, tiene que copiarlas otra vez una por una. Además no hay nada que le avise si se le olvidó marcar la respuesta correcta o si una pregunta se quedó sin opciones, y esos errores se notan hasta que los alumnos ya estan contestando.

3.2 Nuestra propuesta: Cuestiona

Elegimos hacer un lenguaje para describir cuestionarios y le pusimos **Cuestiona**. La idea es que el docente escriba su examen en un archivo de texto normal (con extensión .cst) usando palabras en español, y que la herramienta haga tres cosas con ese archivo:

- **Revisar:** Lee el archivo y dice si está bien escrito. Si hay un error, dice en qué línea está y qué falta, con palabras sencillas.
- **Aplicar:** Muestra el cuestionario en la consola para que un alumno lo conteste y al final le da su calificación.
- **Convertir:** Pasa el mismo cuestionario a HTML, para abrirlo en el navegador, y a JSON, que es un formato que otros programas pueden leer.

Lo consideramos un metalenguaje porque el archivo no es el examen terminado, sino una descripción del examen. Con esa misma descripción se pueden generar varias versiones distintas sin volver a escribir las preguntas. Abajo se ve un primer boceto de cómo se podría ver un archivo. La sintaxis oficial la vamos a definir en el Sprint 1, despues de investigar otros metalenguajes, así que puede cambiar.

```
cuestionario "Repaso de Procesadores de Lenguajes"

seccion "Conceptos básicos"

pregunta opcion "¿Qué es un metalenguaje?" puntos 2
  - "Un formato de imagen"
  * "Un lenguaje que sirve para describir otros lenguajes"
  - "Un sistema operativo"

pregunta vf "Un analizador léxico separa el texto en tokens." puntos 1
  respuesta verdadero

pregunta abierta "¿Qué significan las siglas DSL?" puntos 1
  acepta "lenguaje de dominio especifico", "domain specific language"
```

Figura 1. Boceto preliminar de un archivo .cst (puede cambiar en el Sprint 1).

3.3 Usuarios

- **Docentes:** Son los usuarios principales, los que escriben y corrigen los cuestionarios.
- **Estudiantes:** Contestan el cuestionario ya generado, ya sea en la consola o en la página HTML.
- **Desarrolladores:** Pueden tomar el JSON para usar los cuestionarios en otro sistema.

3.4 Alcance

Al terminar el Sprint 4 queremos tener preguntas de opción múltiple, verdadero o falso y respuesta abierta, con puntos por pregunta y secciones, el validador con mensajes de error, la aplicación en consola y la exportación a JSON y HTML. No vamos a hacer una interfaz gráfica para editar los cuestionarios, ni guardar las respuestas de los alumnos en una base de datos, ni preguntas con imágenes, porque no nos daría tiempo en cuatro semanas y preferimos que lo que sí hagamos funcione bien.

4. Product Backlog inicial

El backlog tiene 16 historias de usuario ordenadas por prioridad. Para estimar usamos **puntos de historia** con la escala 1, 2, 3, 5 y 8. No son horas, son una forma de comparar qué historia cuesta más trabajo que otra. La columna de sprint es solo una sugerencia y se confirma en cada Sprint Planning.

ID	Historia de usuario	Criterios de aceptación	Prior.	Pts.	Sprint
HU-01	Como equipo, quiero investigar metalenguajes existentes (JSON Schema, XML, Regex, ANTLR, SQL) para tomar ideas sin copiar uno.	Documento con al menos 4 metalenguajes revisados. Qué describe cada uno y qué idea nos sirve. Fuentes citadas.	Alta	3	1
HU-02	Como docente, quiero un documento con las reglas del lenguaje para saber qué puedo escribir.	Gramática escrita en EBNF. Un ejemplo por tipo de pregunta. Marcado como versión 0.1.	Alta	5	1
HU-03	Como docente, quiero crear un archivo .cst con el título del cuestionario para empezar a agregar preguntas.	cuestionario "Título" se procesa sin errores. Si falta el título se muestra un error. Se muestra cuántas preguntas hay.	Alta	3	2
HU-04	Como docente, quiero escribir preguntas de opción múltiple y marcar la correcta.	La correcta se marca con * y las demás con -. Error si ninguna es correcta. Mínimo 2 opciones.	Alta	3	2
HU-05	Como docente, quiero escribir preguntas de verdadero o falso para hacer preguntas rápidas.	Se usa respuesta verdadero o respuesta falso. Otro valor se marca como error.	Alta	2	2
HU-06	Como docente, quiero que el programa me diga la línea y el problema cuando me equivoque.	Cada error indica línea y columna. Mensaje en español y sin términos internos. Se muestran todos los errores, no solo el primero.	Alta	5	2
HU-07	Como docente, quiero dejar comentarios en el archivo para recordar de dónde saqué una pregunta.	Lo que va después de # se ignora. Un # dentro de comillas no es comentario.	Media	1	2
HU-08	Como docente, quiero preguntas de respuesta abierta para evaluar definiciones cortas.	Varias respuestas aceptadas con acepta "a", "b". No importan mayúsculas, acentos ni espacios de más.	Alta	3	3
HU-09	Como docente, quiero dar puntos a cada pregunta para que la calificación sea justa.	Se escribe puntos N; si no, vale 1. Se calcula el total. Error si el valor es negativo o no es número.	Media	2	3
HU-10	Como docente, quiero dividir el cuestionario en secciones para organizar los temas.	seccion "Nombre" agrupa las preguntas siguientes. Se muestra el puntaje por sección.	Media	3	3

ID	Historia de usuario	Criterios de aceptación	Prior.	Pts.	Sprint
HU-11	Como estudiante, quiero contestar el cuestionario en la terminal para practicar y ver mi calificación.	Preguntas una por una. Al final, calificación y preguntas incorrectas. Una respuesta inválida se vuelve a preguntar.	Alta	8	3
HU-12	Como desarrollador, quiero el cuestionario en JSON para usarlo en otra aplicación.	Se genera un .json válido. Conserva preguntas, opciones, respuestas y puntos.	Media	3	4
HU-13	Como docente, quiero generar una página HTML del cuestionario para compartirla.	Se abre en el navegador sin instalar nada. Las respuestas correctas no se ven en la página del alumno.	Media	5	4
HU-14	Como docente, quiero escribir una explicación por pregunta para que el alumno entienda su error.	Se escribe con explicacion "texto". Solo se muestra si la respuesta fue incorrecta.	Baja	2	4
HU-15	Como docente, quiero que preguntas y opciones salgan en desorden para evitar que se copien.	Se activa con aleatorio en el encabezado. La correcta se califica bien aunque cambie de lugar.	Baja	2	4
HU-16	Como docente, quiero un manual corto con ejemplos para aprender a usar el lenguaje solo.	Explica cómo instalar y ejecutar. Al menos 5 ejemplos válidos y 5 inválidos con su resultado.	Alta	3	4

Tabla 2. Product Backlog inicial (total: 53 puntos).

Pusimos arriba la investigación y la especificación porque sin tener claras las reglas del lenguaje no podemos programar el analizador. Despues siguen las construcciones básicas y los mensajes de error, que es lo mínimo para que el lenguaje sirva. Las exportaciones y lo de orden aleatorio quedaron al final porque son extras que mejoran el producto pero no son necesarias para que funcione.

5. Definition of Done

La Definition of Done, es la lista de condiciones que tiene que cumplir una historia para decir que está terminada. Si falta una sola, la historia no cuenta como hecha en la Sprint Review. La nuestra es esta:

- **Código funcional:** Hace lo que piden los criterios de aceptación y se ejecuta sin errores.
- **Pruebas:** Tiene pruebas automáticas con pytest que revisan los criterios, y todas las pruebas del proyecto pasan.
- **Documentación actualizada:** Si cambia el lenguaje se actualiza la especificación, y si cambia la forma de usar la herrameinta se actualiza el manual o el README.
- **Revisión por un compañero:** El otro integrante revisa el cambio en un Pull Request de

GitHub y lo aprueba antes de unirlo a la rama principal.

- **Ejemplos:** Si se agrega algo nuevo al lenguaje, se agrega por lo menos un ejemplo válido y uno inválido.
- **Tablero al día:** La tarjeta de la historia está en la columna de terminado.

En las historias que solo son documentos, como la investigación o la especificación, las condiciones de código, pruebas y ejemplos se cambian por esto: el documento está en el repositorio y otro integrante ya lo revisó.

6. Plan tentativo de sprints

Sprint	Objetivo	Inicio	Fin
Semana 0	Preparación: visión, backlog, DoD y roles	12/11/2026	12/11/2026
Sprint 1	Fundamentos y especificación v0.1	16/11/2026	22/11/2026
Sprint 2	Núcleo: analizador y validador	23/11/2026	29/11/2026
Sprint 3	Ampliación y validación con la docente	30/11/2026	06/12/2026
Sprint 4	Cierre, documentación y demo final	07/12/2026	13/12/2026

Tabla 3. Calendario tentativo; las fechas se confirman en cada Sprint Planning.

7. Repositorio y herramientas

Todo el proyecto está en el repositorio **github.com/Ebrahilok/metalenguaje**. Por ahora tiene el README, los documentos de esta entrega y plantillas para las actas de Planning, Daily, Review y Retrospective, el sprint backlog y los datos del burndown, para que en cada sprint solo tengamos que llenarlas. Cada integrante va a subir sus cambios con su propia cuenta, así el historial de commits muestra lo que hizo cada quien.

Lo vamos a programar en **Python**, porque el código se lee fácil, lo que ayuda a poder explicarlo en la demo, y tiene pytest para las pruebas. Como tablero Scrum vamos a usar **GitHub Projects**, que se crea en el Sprint Planning del Sprint 1 con una tarjeta por cada historia del backlog.

8. Uso de herramientas de IA

El enunciado permite usar inteligencia artificial siempre que se declare. En esta entrega usamos **Claude** (de Anthropic) como apoyo para redactar la visión, el backlog, la Definition of Done y el README, y para preparar la estructura del repositorio. Nosotros elegimos el tema de cuestionarios, los roles y el alcance, y revisamos todo antes de subirlo. Esta declaración también está en el repositorio, en docs/uso-de-ia.md, y la vamos a actualizar en cada sprint donde volvamos a usar IA.

9. Conclusión

En conclusión, con esta Semana 0 ya tenemos claro qué vamos a construir y en qué orden. Escribir los criterios de aceptación desde ahora nos va a servir mucho, porque cada historia ya dice exactamente qué tiene que pasar y eso mismo es lo que vamos a revisar en las pruebas. El siguiente paso es el Sprint Planning del Sprint 1, donde vamos a investigar otros metalenguajes y escribir la primera versión de la gramática.